FASTA Generation Methodology & Planned Accessions Rationale

Bioinformatics Course — MS Data Science — Fall 2026 | PUCIT

Student Name: Muhammad Imran Mian
Roll Number: MSDSF25A007
Instructor: Dr. Shahid Manzoor
Program / Uni: MS Data Science (PUCIT)

1. Planned vs. Random Accessions

The 5 nucleotide accessions utilized in this assignment were strictly planned and curated, rather than chosen at random.

In bioinformatics workflows, querying random accessions poses severe reliability issues:
• Many NCBI nucleotide entries are WGS/CON records (segmented master records or contig pointers) that do not contain an inline sequence (ORIGIN block).
• Many entries are entire mammalian chromosomes spanning hundreds of millions of base pairs, which would cause network timeouts, excessive memory consumption, and massive disk writes.
• Random records fail to provide a balanced biological distribution for benchmarking sequence statistics like GC content.

Rationale Behind the 5 Chosen Accessions

Accession	Biological Entity	Type	Length	GC%	Methodological Rationale
NC_045512.2	SARS-CoV-2	ssRNA Virus	29,903 bp	37.97%	Global reference genome; mid-length viral genome with low-to-moderate GC content.
NM_000518.5	Homo sapiens HBB mRNA	Eukaryotic mRNA	628 bp	51.27%	Classic human clinical model; validates handling of shorter coding transcripts with balanced GC.
NR_046235.3	Homo sapiens 45S pre-rRNA	Non-coding RNA	13,357 bp	72.55%	Critical benchmark for high-GC structural RNA parsing and GC-content skew detection.
NC_001416.1	Escherichia phage Lambda	dsDNA Phage	48,502 bp	49.86%	Canonical molecular biology model organism; demonstrates processing of larger genomes (~48.5

					kb).
AY278741.1	SARS-CoV Urbani	Complete Genome	29,727 bp	40.80%	Historical comparative reference to SARS-CoV-2; verifies repeatability on coronavirus architecture.

2. End-to-End Generation Methodology

The generated .fasta files were not downloaded directly as pre-packaged FASTA files from a web browser. Instead, our custom script retrieve_sequences_plain.py executed a complete bioinformatics extraction and conversion pipeline using only Python standard library modules (urllib, re, ssl, time, os).

Step A: Manual HTTP Request Construction

For each accession, an E-utilities request is constructed targeting the NCBI Entrez REST interface:
https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=nucleotide&id=<ACCESSION>&rettype=gb&retmode=text
The script dispatches an HTTP GET request using urllib.request.Request with an academic User-Agent header and handles SSL verification gracefully.

Step B: Metadata Parsing via Regular Expressions

The GenBank flat-file format stores taxonomic metadata in structured multi-line headers. The organism scientific name is extracted using:
re.search(r"^\s*ORGANISM\s+(.+)$", genbank_text, re.MULTILINE)

Step C: Sequence Extraction and Sanitization

GenBank records store the nucleotide sequence at the end of the file between the ORIGIN header and the record-terminating // delimiter. The raw text interleaves base pairs with 60-character line lengths, 9-digit base counters, and whitespace.
1. Block extraction: re.search(r"\bORIGIN\s*\n([\s\S]*?)(?:\n\s*//|\Z)", genbank_text)
2. Noise stripping: Line numbers (\d) and whitespace/newlines (\s) are stripped using regex, and the resulting string is converted to uppercase: re.sub(r"[\d\s]", "", raw_block).upper()

Step D: Standard-Compliant FASTA Formatting

Standard FASTA format requires a single description line beginning with '>' followed by the identifier and organism, and sequence lines wrapped at a uniform width (the assignment specification mandates 70 characters per line).
The script wraps the sanitized sequence string into 70-character chunks:
'\n'.join(sequence[i:i + 70] for i in range(0, len(sequence), 70))
The formatted content is then written to <accession>.fasta.

Step E: Direct String Mathematics (Length and GC Content)

Without relying on external bioinformatics libraries, sequence length and Guanine-Cytosine (GC) ratio are calculated directly from the clean sequence string:
GC% = ((count('G') + count('C')) / sequence_length) * 100

Step F: Rate-Limiting & Fault Tolerance

To comply with NCBI's fair-use API guidelines (< 3 unauthenticated requests per second), the script executes a mandatory pause between queries (time.sleep(0.4)). Network operations are wrapped in try/except blocks to ensure a transient error on one accession does not terminate batch processing.